

Software Requirements Specification (SRS)

Hospital Management System (Microservices Architecture)

1. Introduction

1.1 Purpose

The objective of this document is to provide a comprehensive and precise description of all functional and non-functional requirements for the Hospital Management System. This system is designed to streamline interactions between patients and doctors while organizing financial and administrative operations using a microservices architecture.

1.2 Scope of the System

The system covers seven primary microservices:

- 1. Auth Service (Module 1):** Manages registration, login, verification, and JWT token issuance.
- 2. Patient Service (Module 2):** Manages demographic data and medical history for patients.
- 3. Doctor Service (Module 2):** Manages professional data for doctors (specialization, experience, consultation fee, approval status).
- 4. Appointment Service (Module 3):** Handles booking, rescheduling, and cancellation of appointments.
- 5. Billing Service (Module 4):** Calculates fees and processes electronic payments.
- 6. API Gateway:** A single entry point for all client requests.
- 7. Eureka Server:** Service discovery registry.
- 8. Monitoring Module (AOP - Module 6):** Tracks operations and records security logs.

1.3 Technology Stack

- **Spring Boot 3.x**
- **Spring Cloud (Eureka, Gateway)**
- **Spring Data JPA**
- **MySQL Database**

- **Docker & Docker Compose**
- **Maven**

1.4 Project Structure

hospital-system/

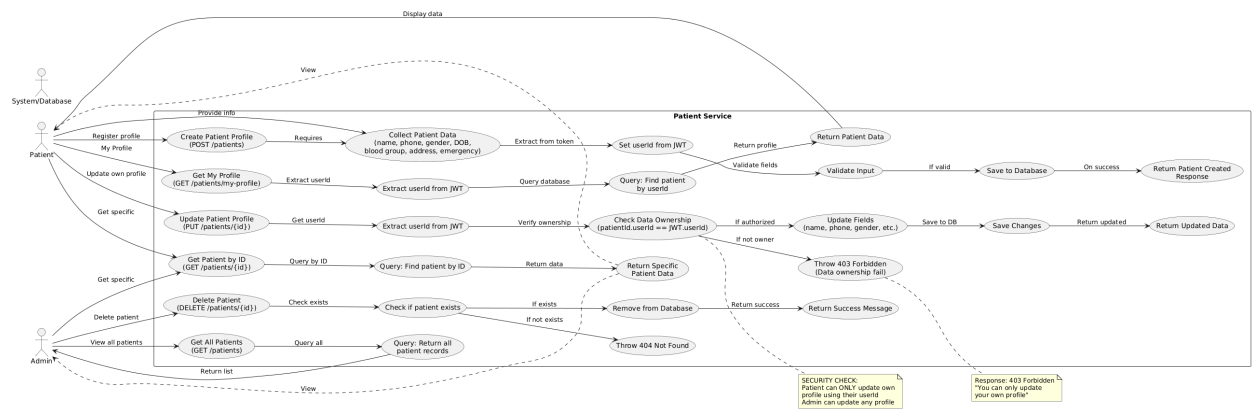
```
|— eureka-server/  
|— api-gateway/  
|— auth-service/  
|— patient-service/  
|— doctor-service/  
|— appointment-service/  
|— billing-service/  
|— mysql/  
|— docker-compose.yml  
└─ hms-frontend/
```

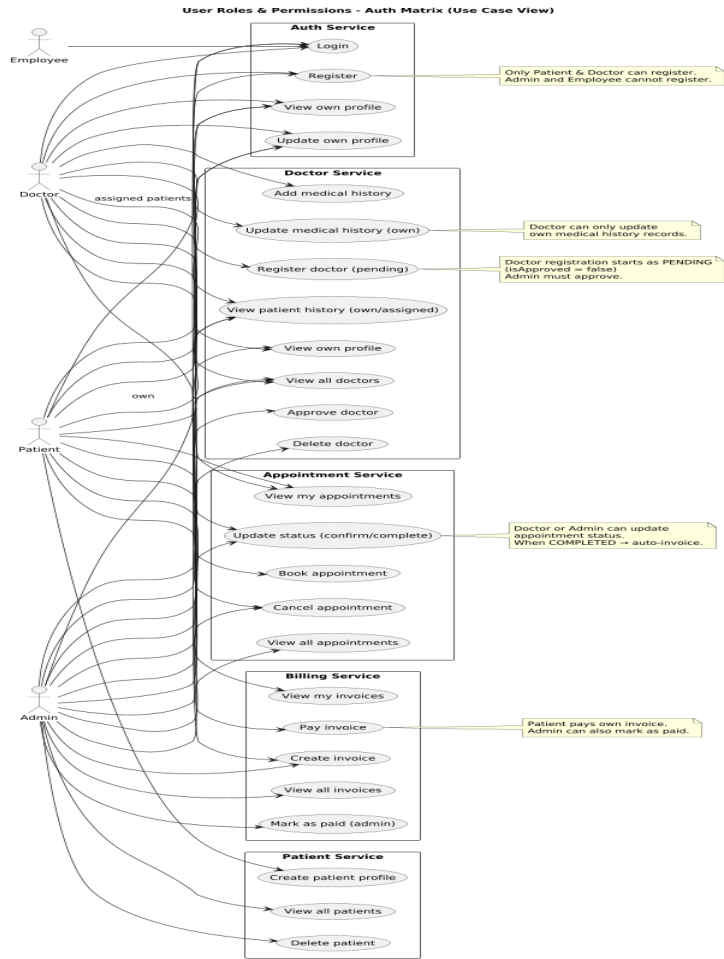
Microservices Communication

- **Service Discovery:** All services register with Eureka Server.
- **API Gateway:** Routes all client requests to appropriate services.
- **Feign Client:** Used for synchronous communication between services.

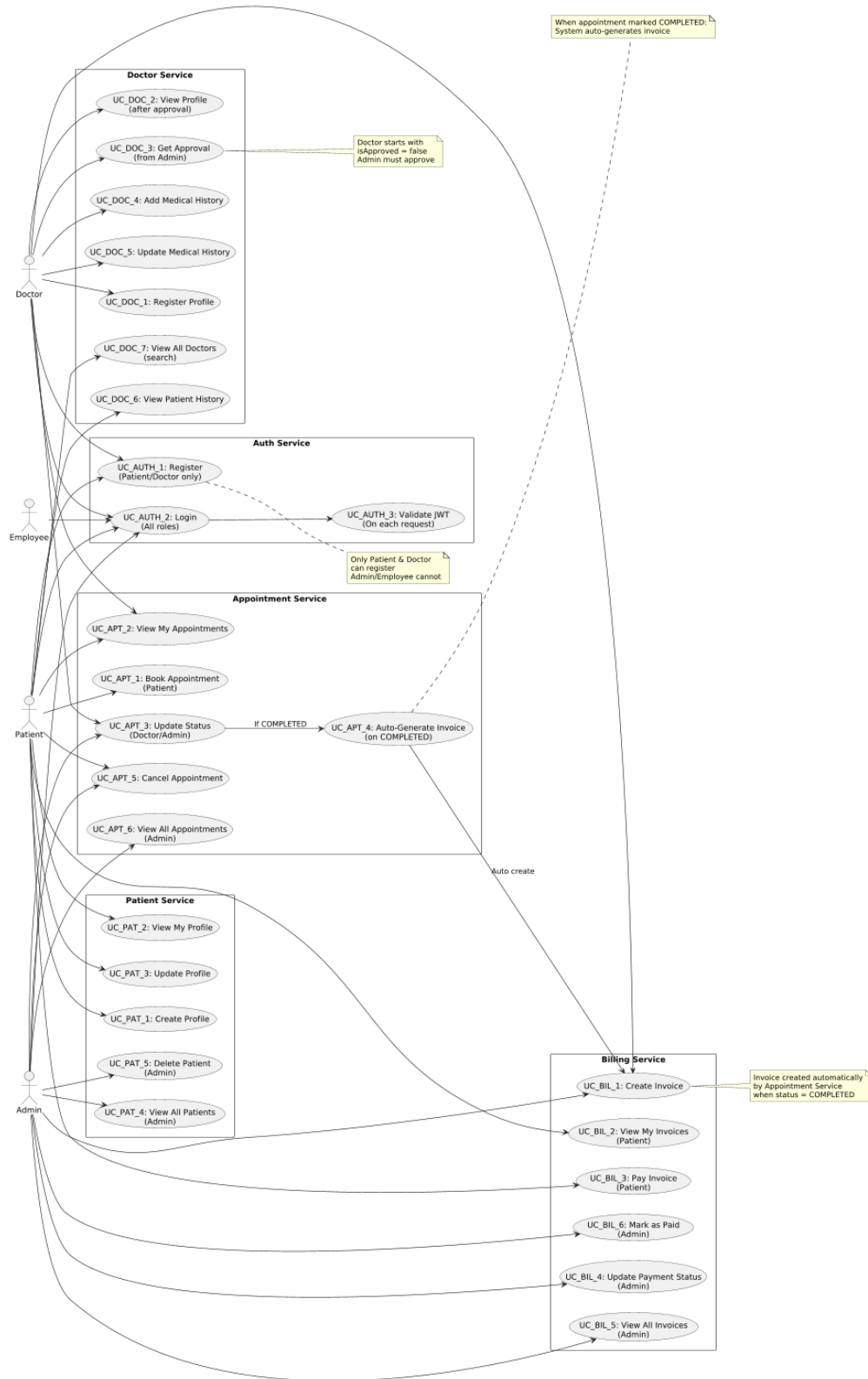
2. Use Case Diagrams

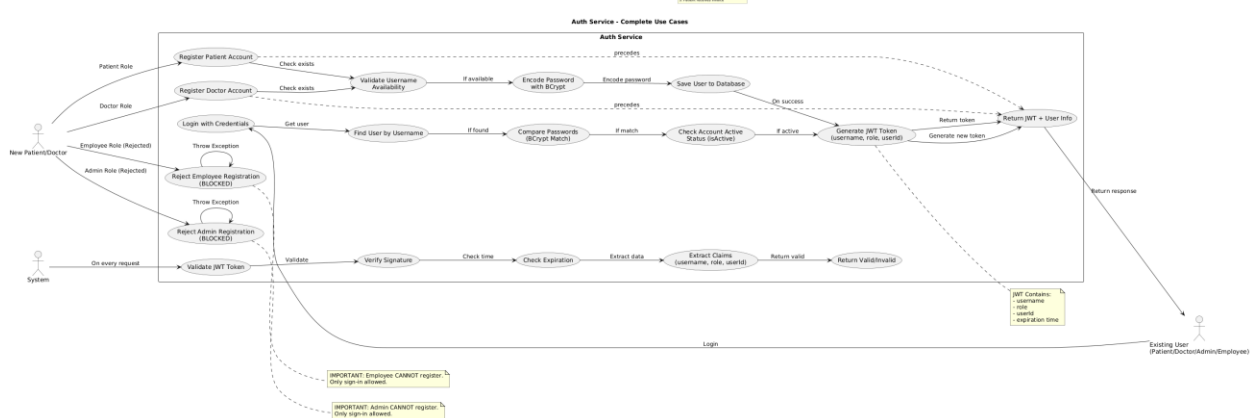
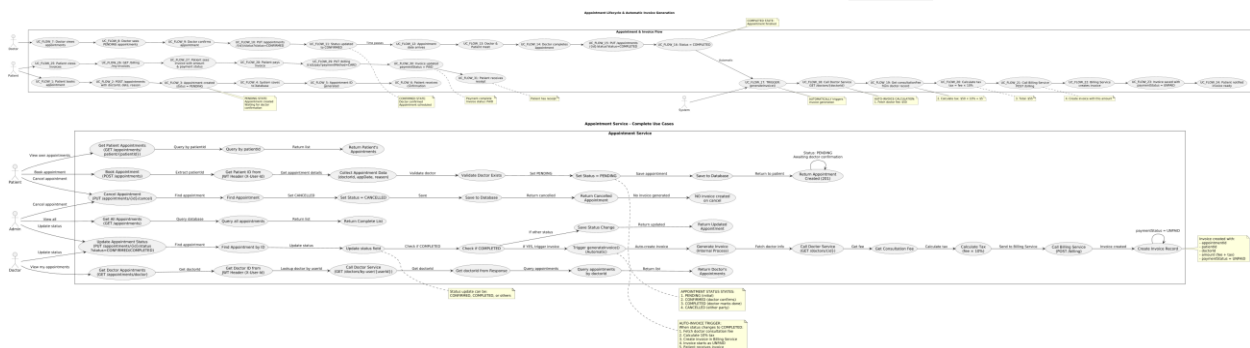
Patient Service - Complete Use Cases

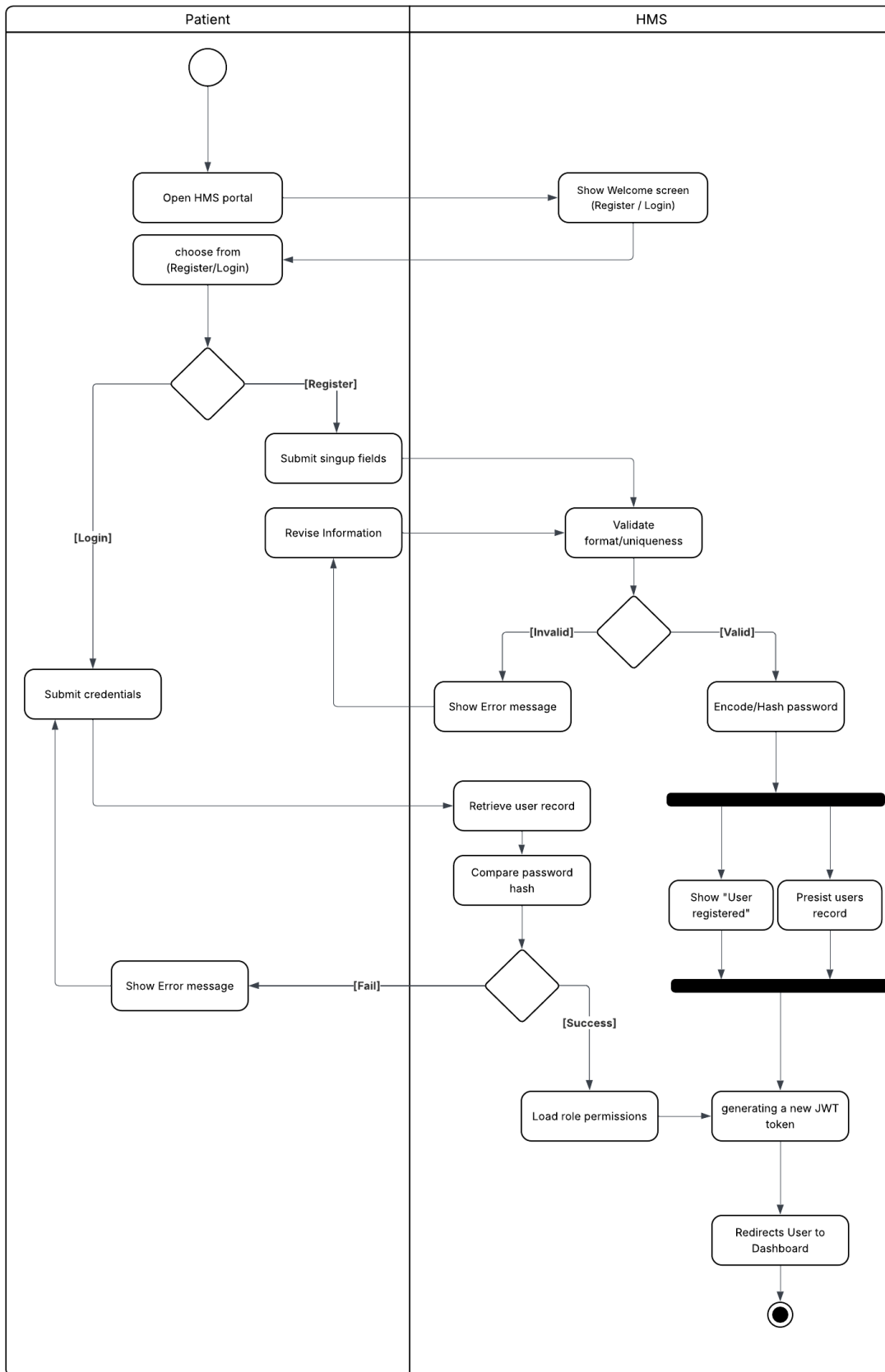


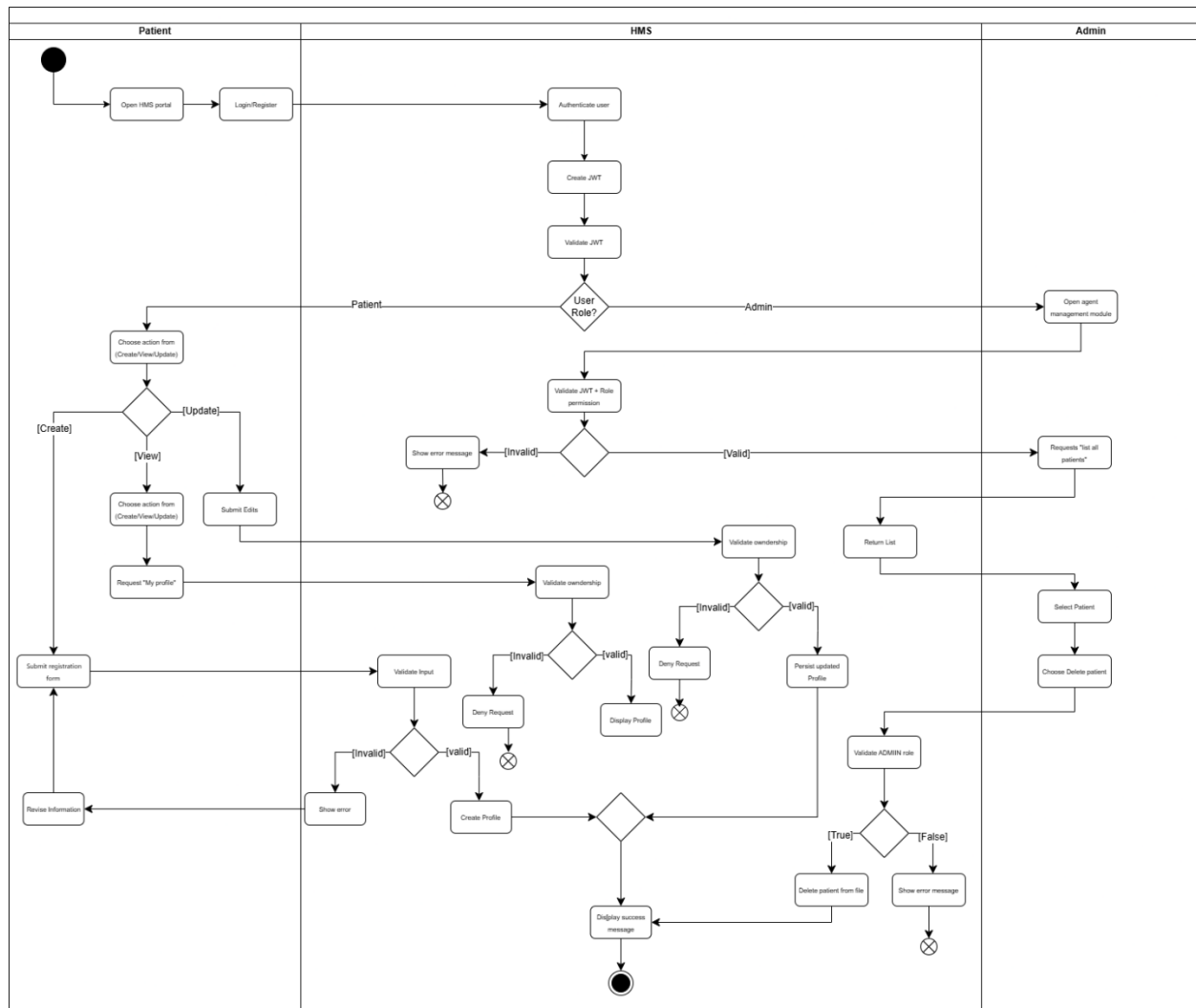


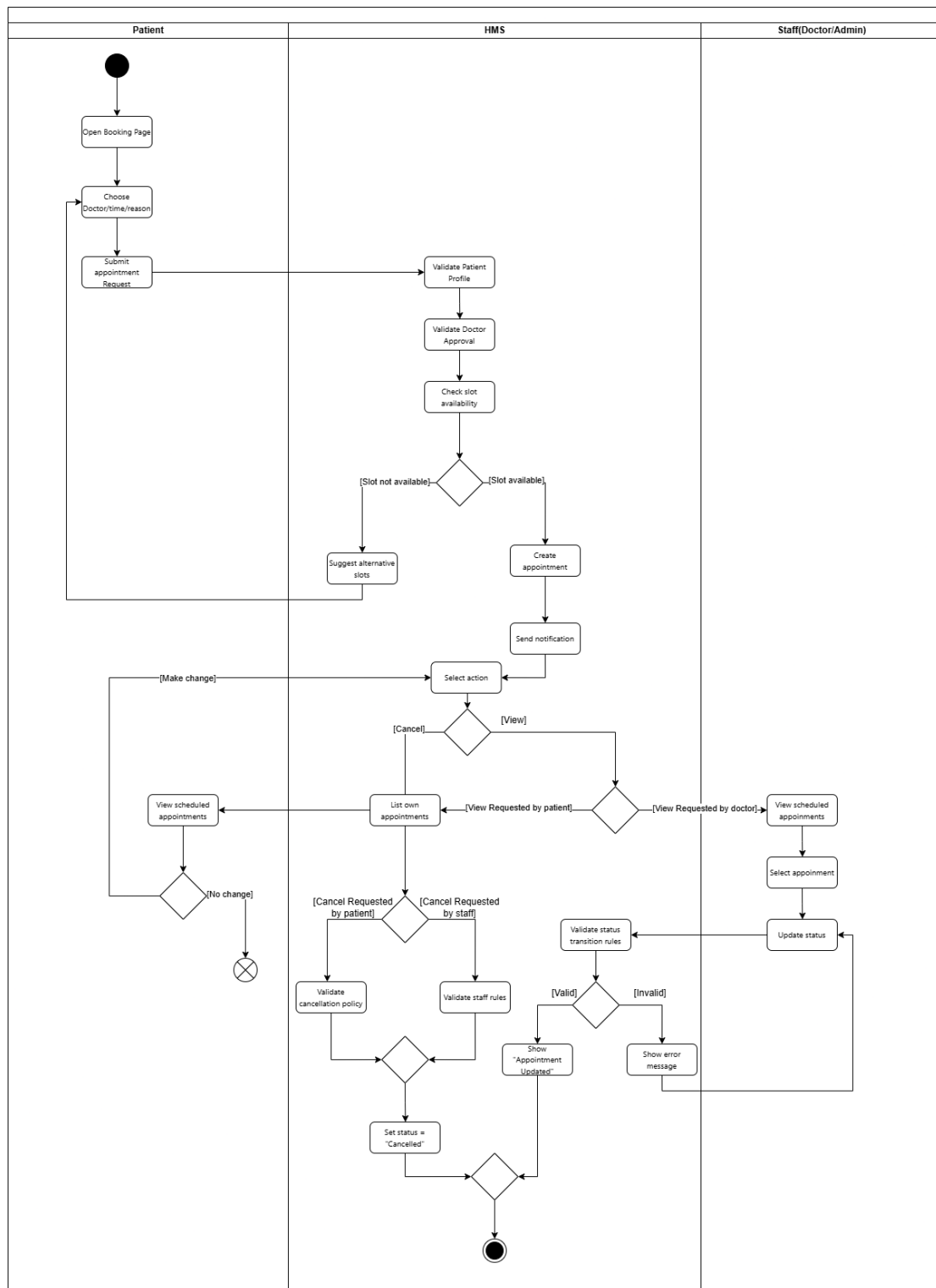
Hospital Management System - Complete Overview

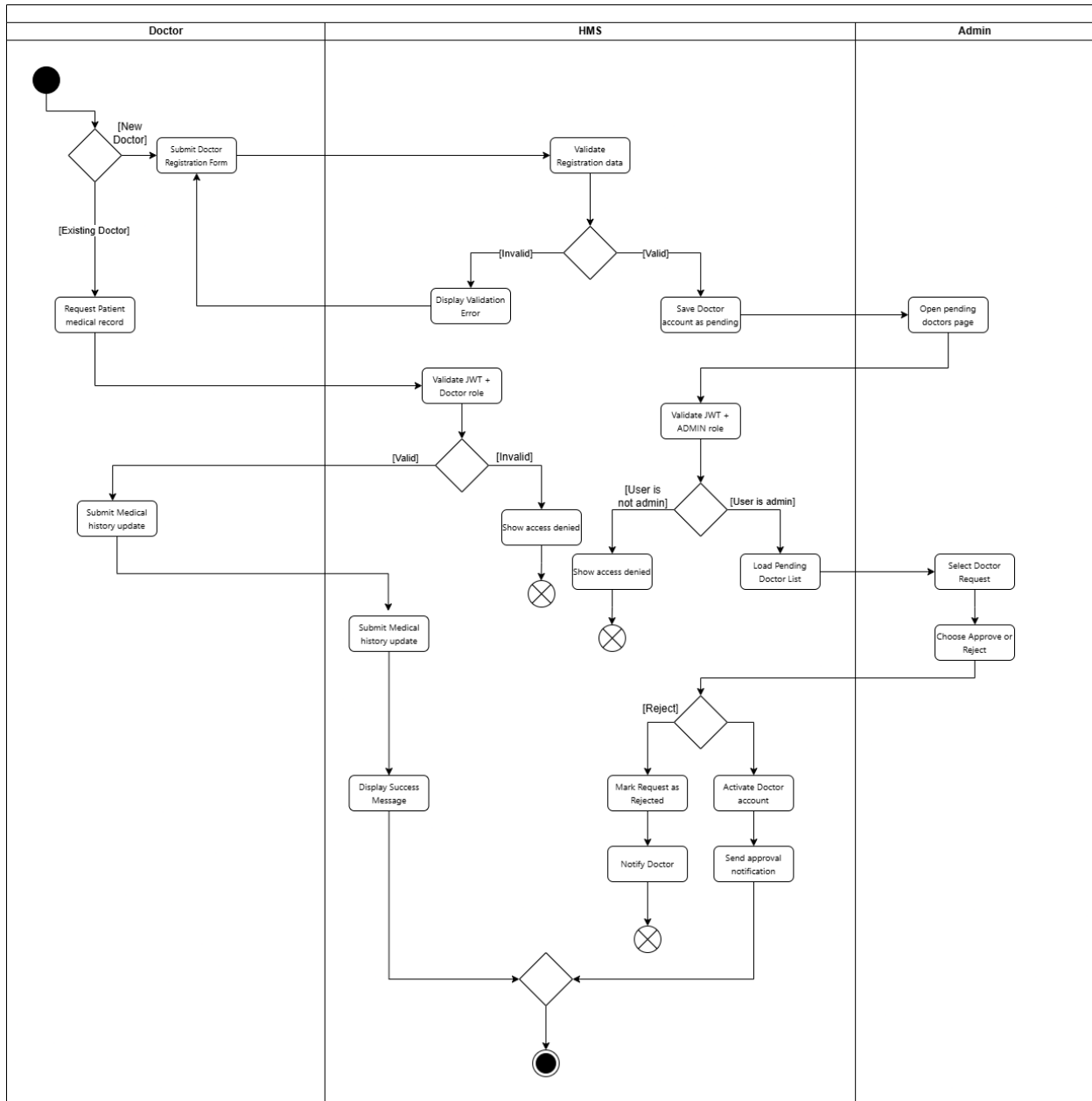


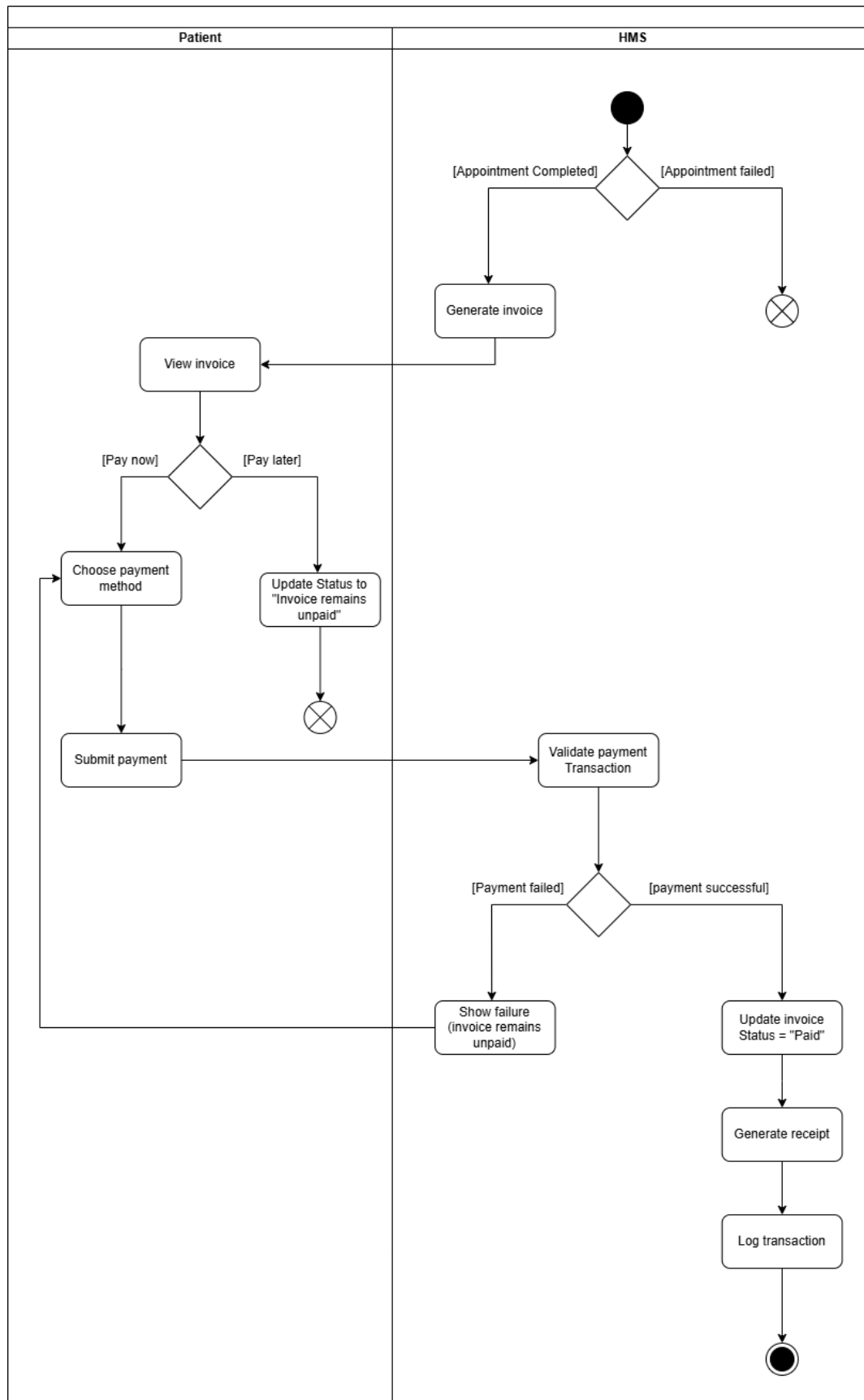












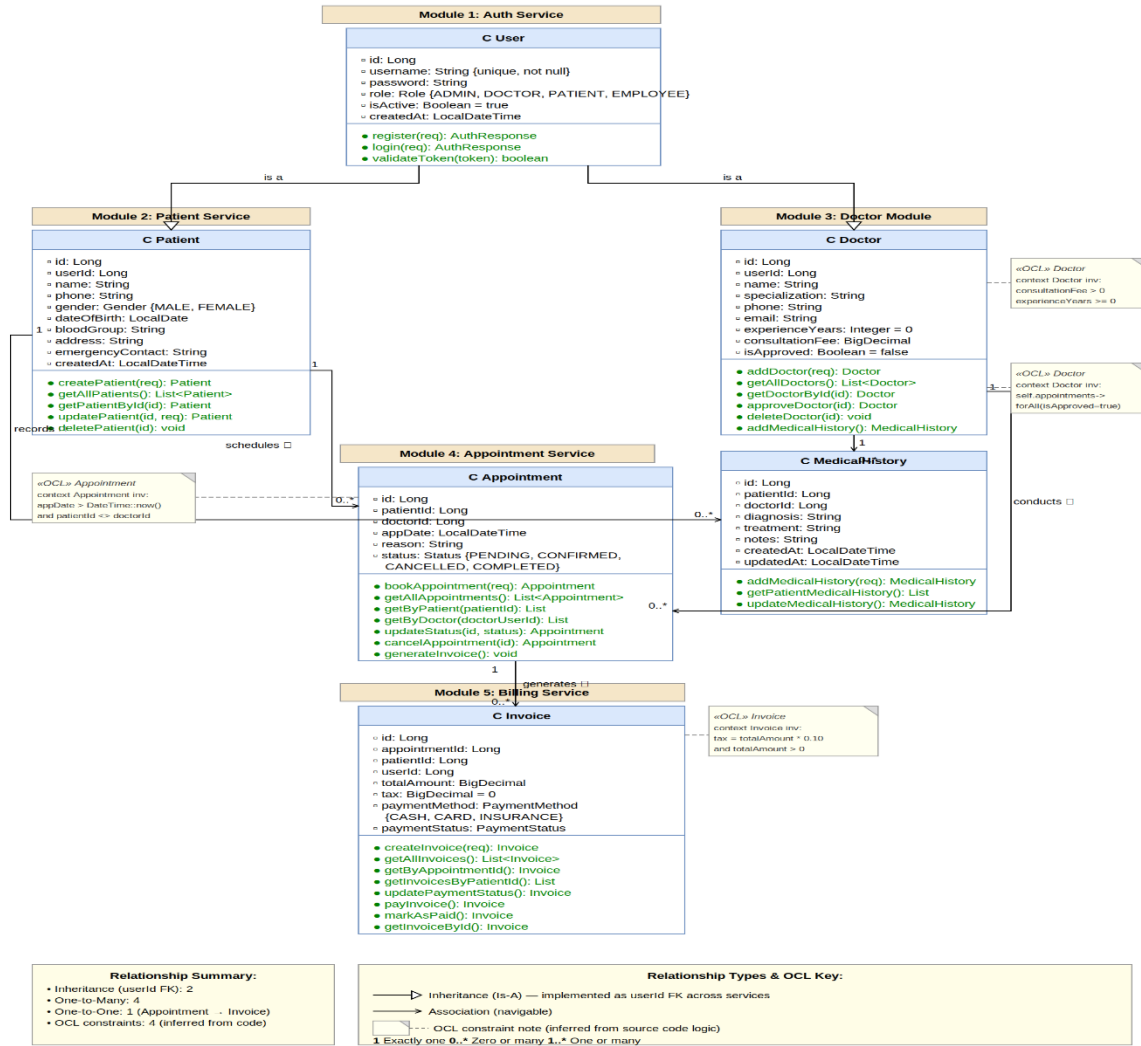
- **Doctor Approval:** Admin approves → isApproved=true.
- **Booking:** Check availability → Confirm or Reject.
- **Billing:** Calculate fees → Payment → Status=Paid.

4. Class Diagram

Primary entities: User, Patient, Doctor, Appointment, Invoice, MedicalHistory.

Relationships:

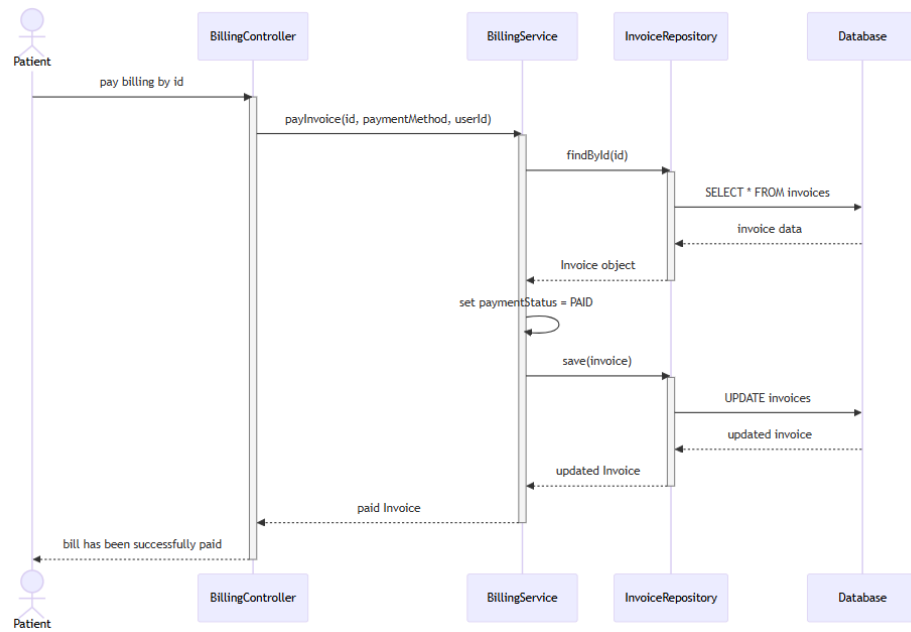
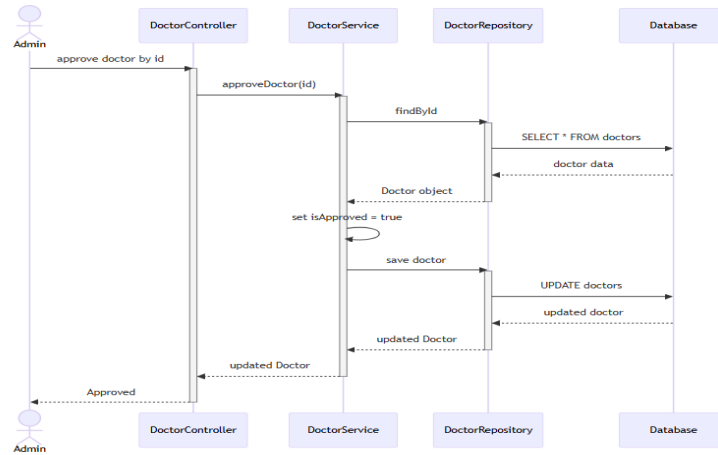
- **Doctor (1) to (M) Appointment**
- **Patient (1) to (M) Appointment**
- **Patient (1) to (M) MedicalHistory**
- **Appointment (1) to (1) Invoice**

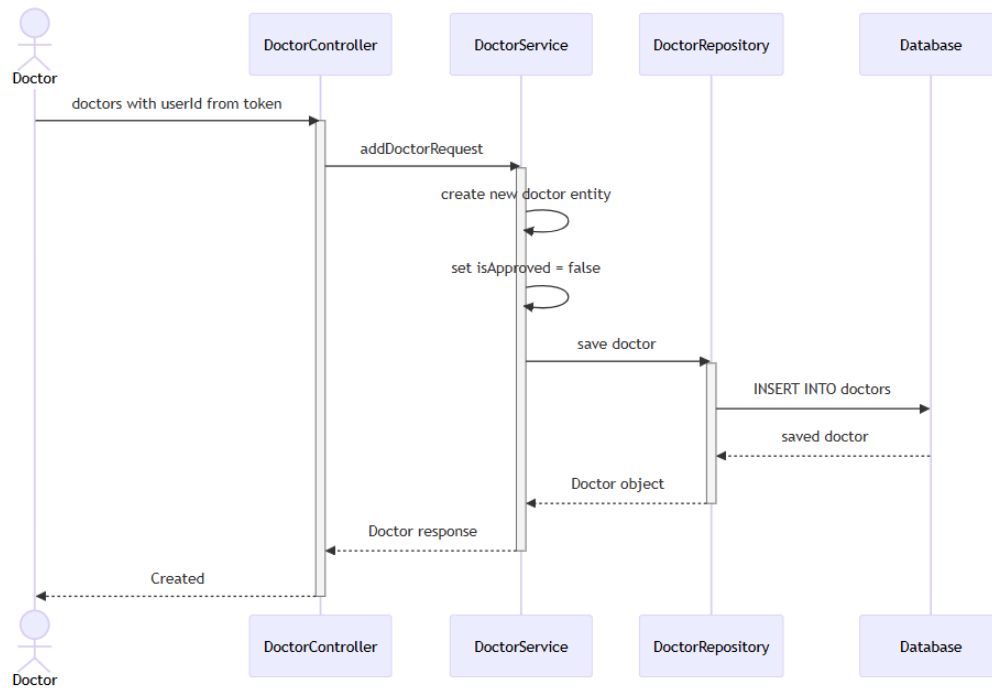
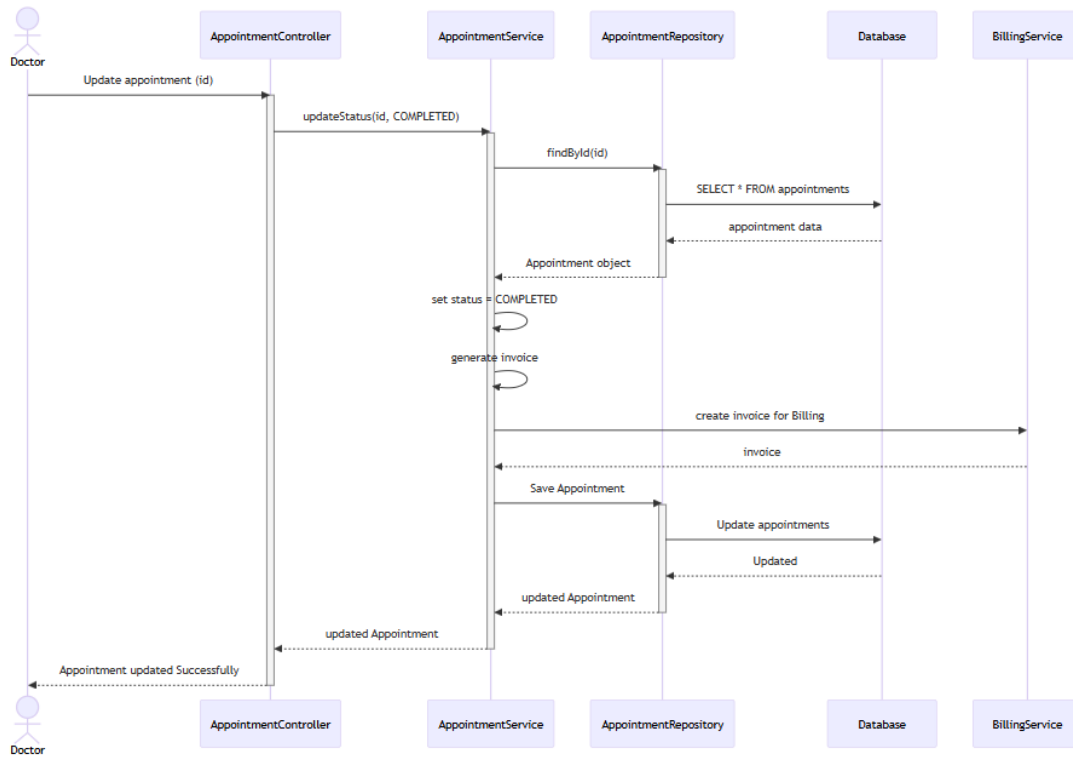


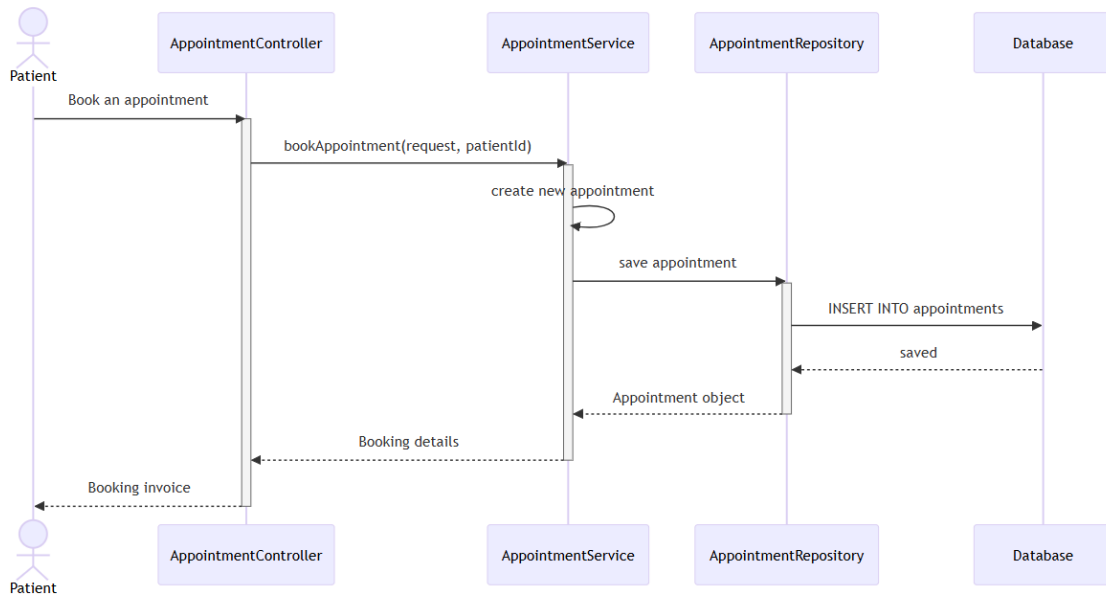
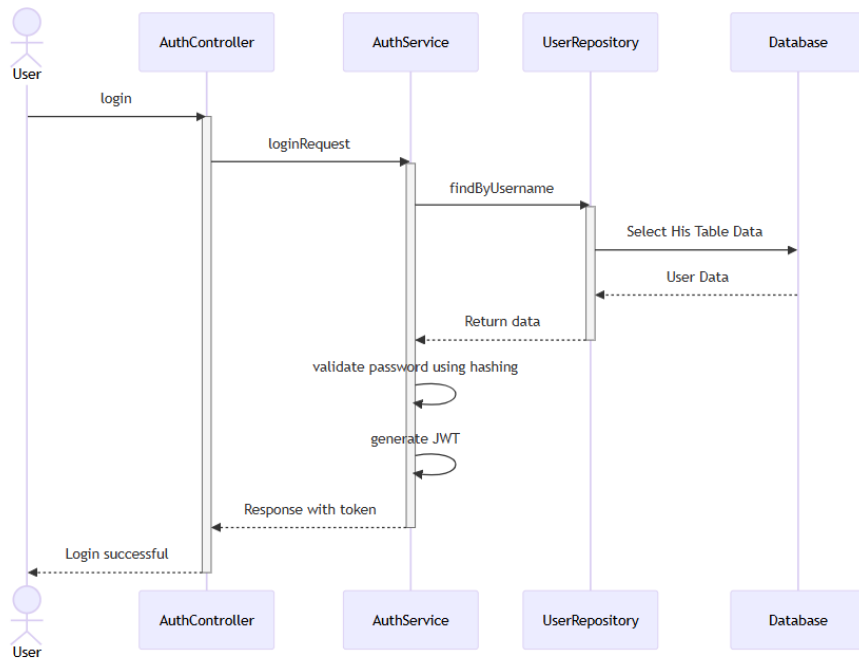
5. Sequence Diagrams

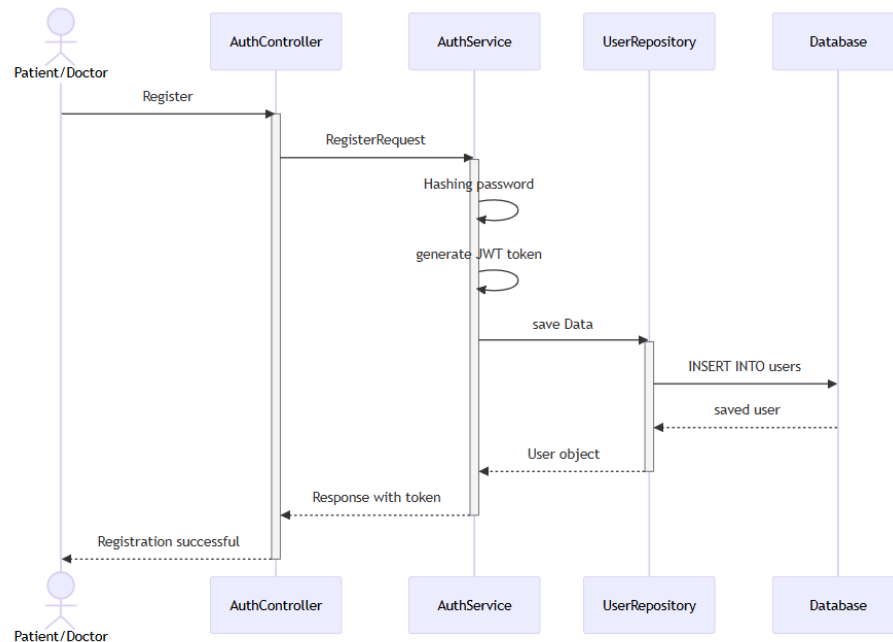
Communication flow: User Interface -> API Gateway -> Service Layer -> Database.

- **Payment:** User -> Gateway -> Billing Service -> Database -> Confirmation.
- **Booking:** Patient -> Gateway -> Appointment Service -> Doctor Service (Feign) -> Database -> Confirmation.
- **Doctor Approval:** Admin -> Gateway -> Doctor Service -> Database -> Response.









6. Design Patterns Used

Design Pattern	Location in Code	Type
Singleton	LoggingAspect.java (Logger)	Creational
Proxy	LoggingAspect.java (@Around)	Structural
Interceptor	RoleInterceptor.java (preHandle)	Behavioral
Front Controller	GlobalExceptionHandler.java	Behavioral
Factory	GlobalExceptionHandler.java (ResponseEntity)	Creational
Dependency Injection	All services (Constructor injection)	Structural
Repository	DoctorRepository.java (JpaRepository)	Data Access

7. Docker & Deployment

7.1 Docker Configuration

docker-compose.yml example:

services:

mysql:

image: mysql:8.0

ports: - "3306:3306"

eureka-server:

build: ./eureka-server

ports: - "8761:8761"

api-gateway:

build: ./api-gateway

ports: - "8080:8080"

7.2 Docker Commands

docker-compose up -d (Start all services)

docker-compose down (Stop all services)

docker ps (View running containers)

docker logs <container> (View service logs)

8. Non-Functional Requirements

- Security: Password encryption (BCrypt) and JWT tokens.
- Availability: 99.9% uptime.
- Usability: Simple interfaces for elderly patients.
- Performance: Execution time logged via AOP.
- Scalability: New service instances can be added without downtime.

9. Functional Requirements

1. User Authentication & Authorization (Auth Service)

FR-1.1: The system shall allow users to register by choosing a role (Patient or Doctor) and providing valid credentials.

FR-1.2: The system shall authenticate users via email and password, issuing a secure JWT token.

FR-1.3: The system shall provide a password recovery mechanism via email.

FR-1.4: The Admin shall have the authority to manage (Create, Update, Delete) user accounts and roles.

2. Appointment Management (Appointment Service)

FR-2.1: The system shall allow Patients to search for doctors based on name, specialty, or department.

FR-2.2: The system shall display real-time availability of doctor time slots.

FR-2.3: The system shall validate slot availability before confirming a booking to prevent double-booking.

FR-2.4: The system shall allow users to reschedule or cancel appointments, automatically releasing the time slot.

3. Medical Profile & Directory (Patient & Doctor Service)

FR-3.1: Doctors shall be able to manage their profiles, including specialties, department, and contact info.

FR-3.2: Patients shall be able to view their own profile and medical history.

FR-3.3: The system shall maintain a searchable directory of all registered and verified doctors (isApproved = true).

FR-3.4: Admin must approve new doctors before they appear in the directory.

4. Billing & Financial Transactions (Billing Service)

FR-4.1: The system shall automatically generate an invoice after a medical consultation is completed.

FR-4.2: The system shall calculate fees including base consultation rates and any additional charges.

FR-4.3: The system shall support multiple payment methods and update the payment status to "Paid" upon success.

FR-4.4: The system shall generate and display a digital receipt for every transaction.

5. System Auditing & Logging (AOP)

FR-5.1: The system shall use Aspect-Oriented Programming (AOP) to log all user activities and API requests.

FR-5.2: The system shall record and log the execution time of each operation for performance monitoring.

FR-5.3: The Admin shall be able to view audit logs to monitor system security and performance.

6. Service Discovery & Gateway (Microservices Infrastructure)

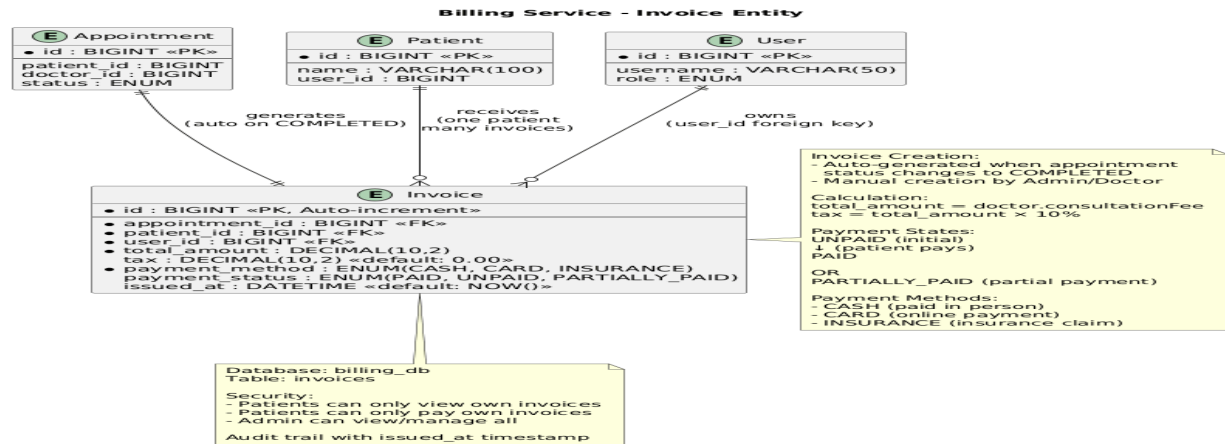
FR-6.1: All services must register with Eureka Server on startup.

FR-6.2: API Gateway must route all external requests to the appropriate internal

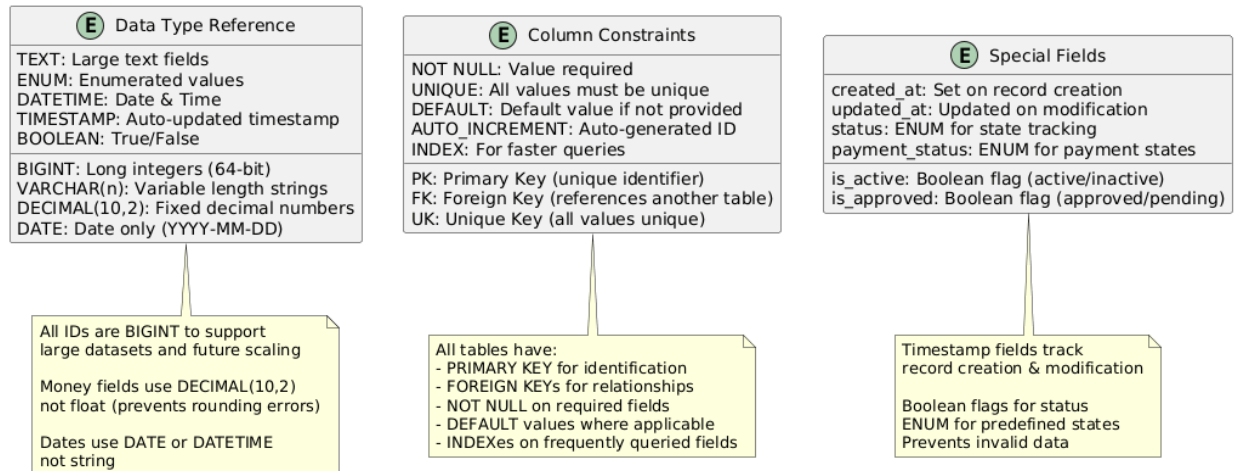
service.

FR-6.3: Services must communicate with each other using Feign Clients.

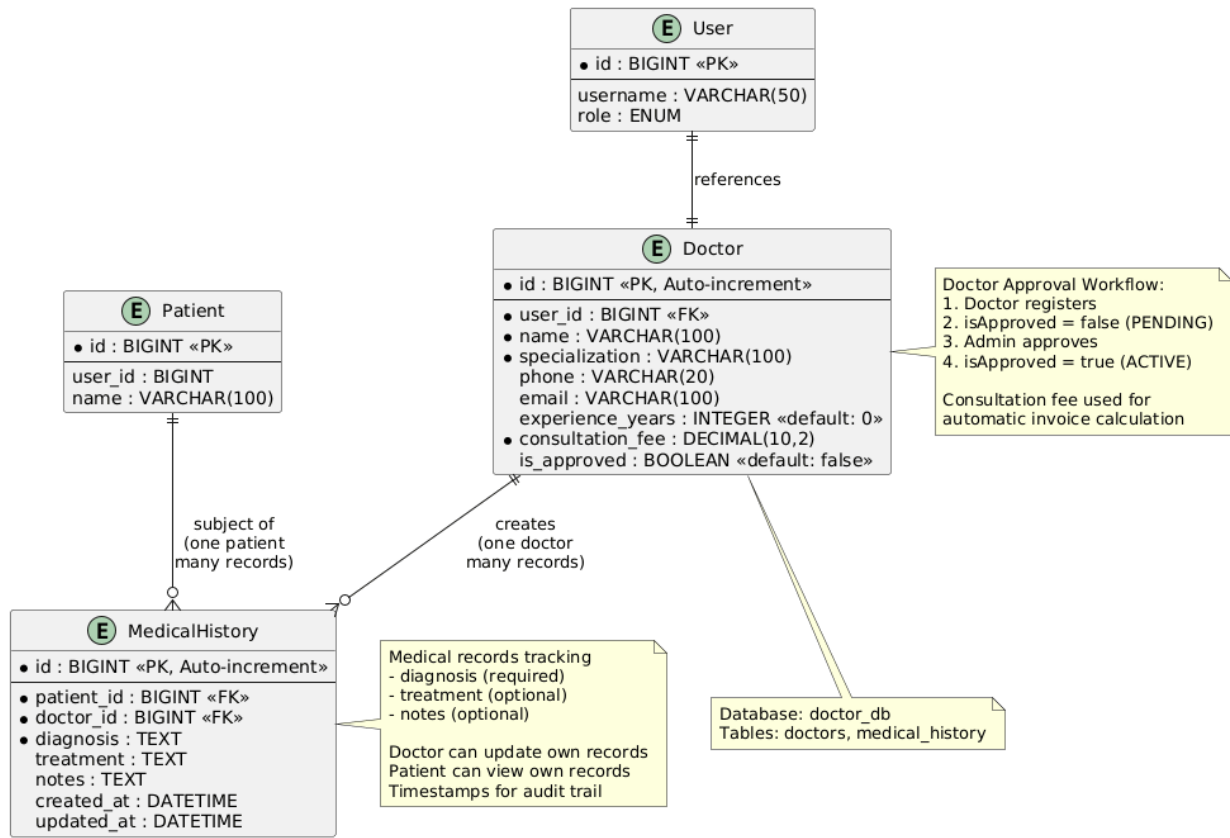
10. ERD Diagram



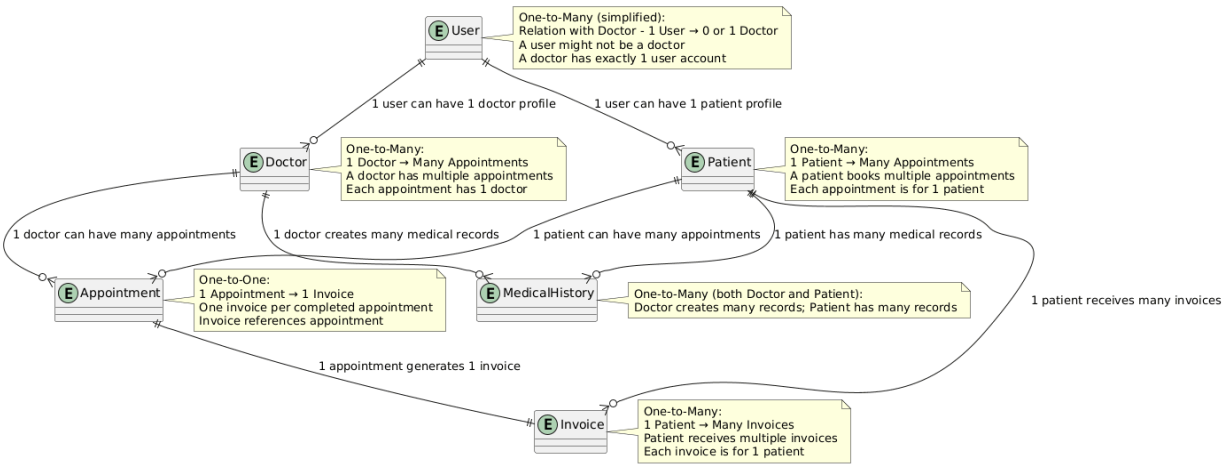
HMS Database - Data Types & Constraints



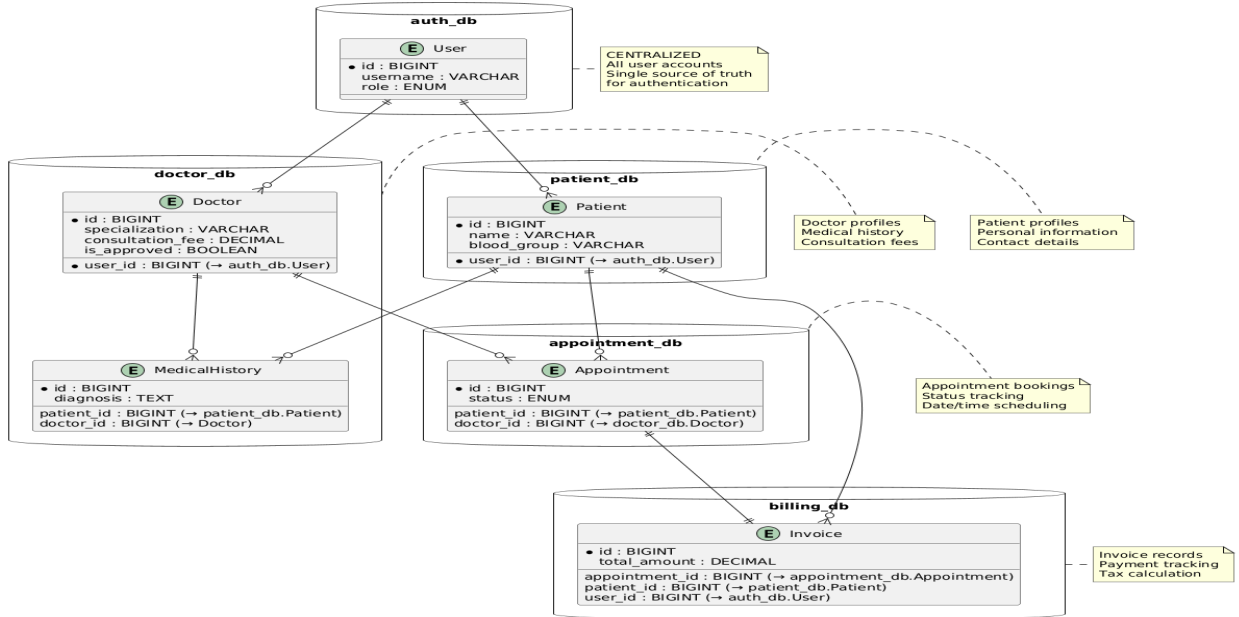
Doctor Service - Doctor & Medical History Entities



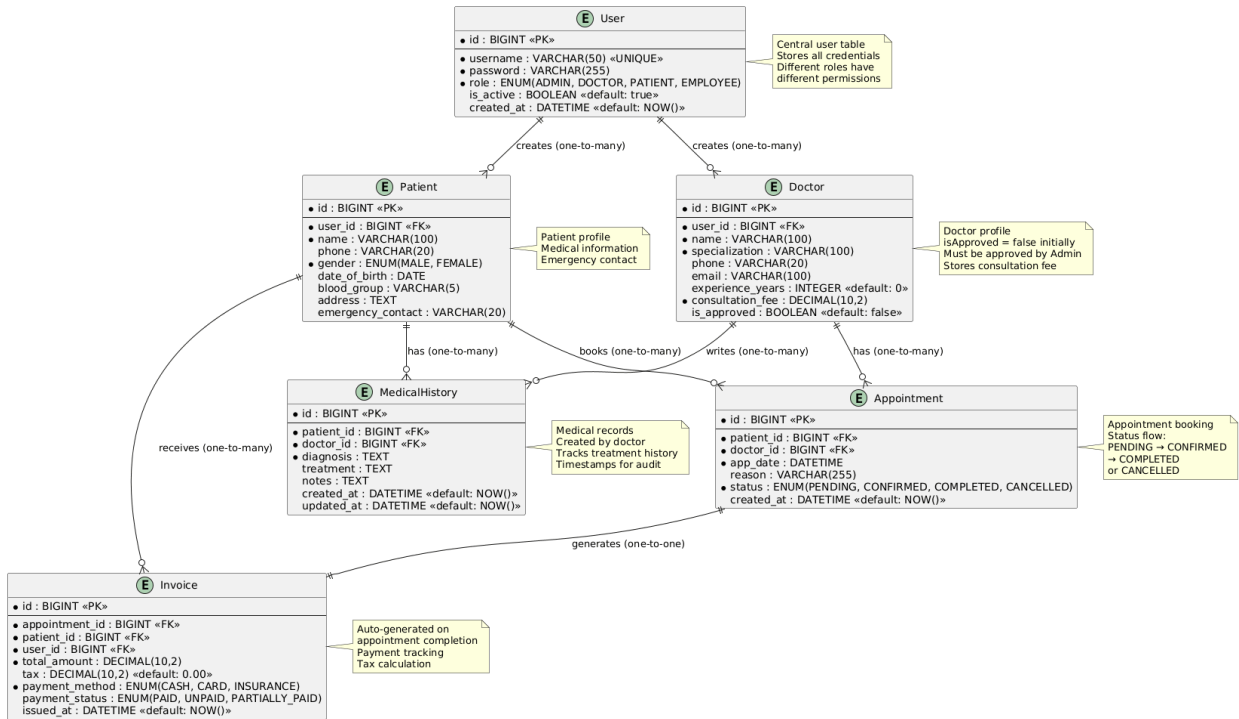
HMS Database - Relationship Cardinality



HMS Database - Cross-Service Relationships



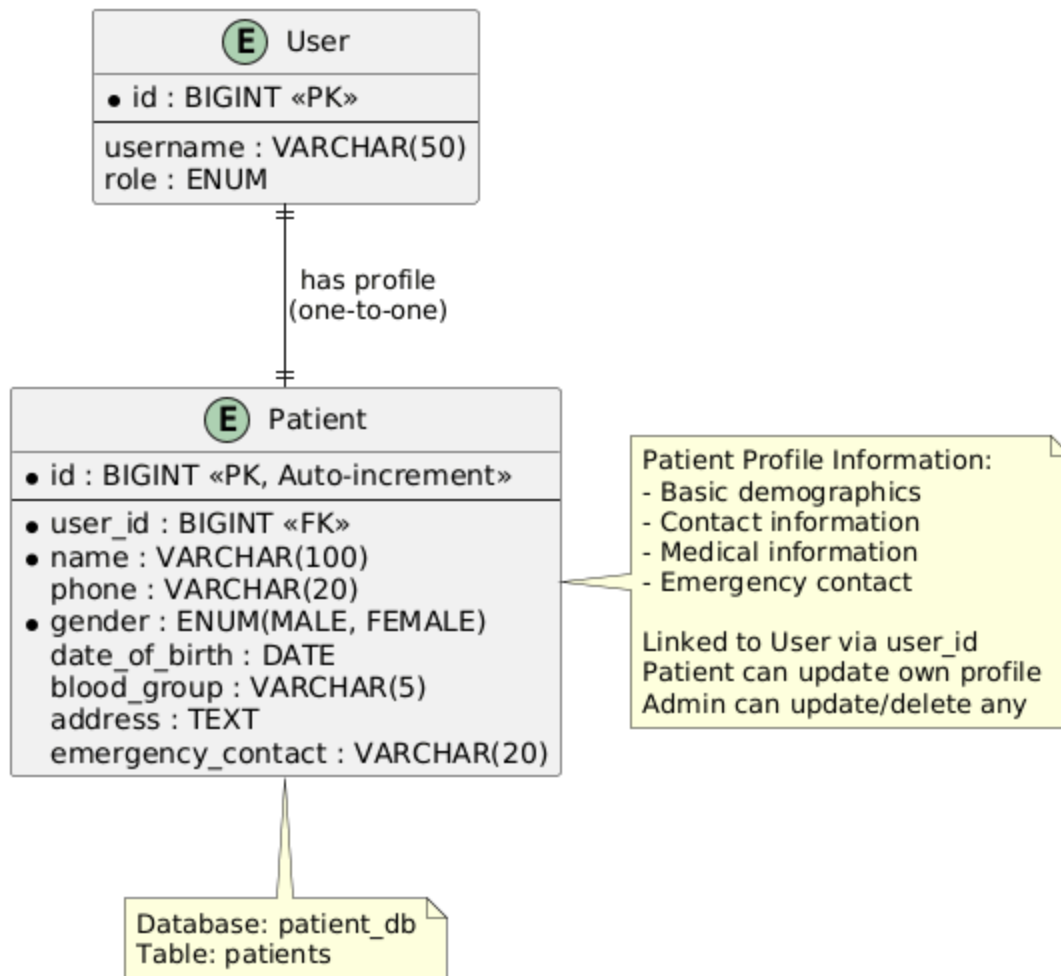
HMS Database - Entity Relationship Diagram (Complete)



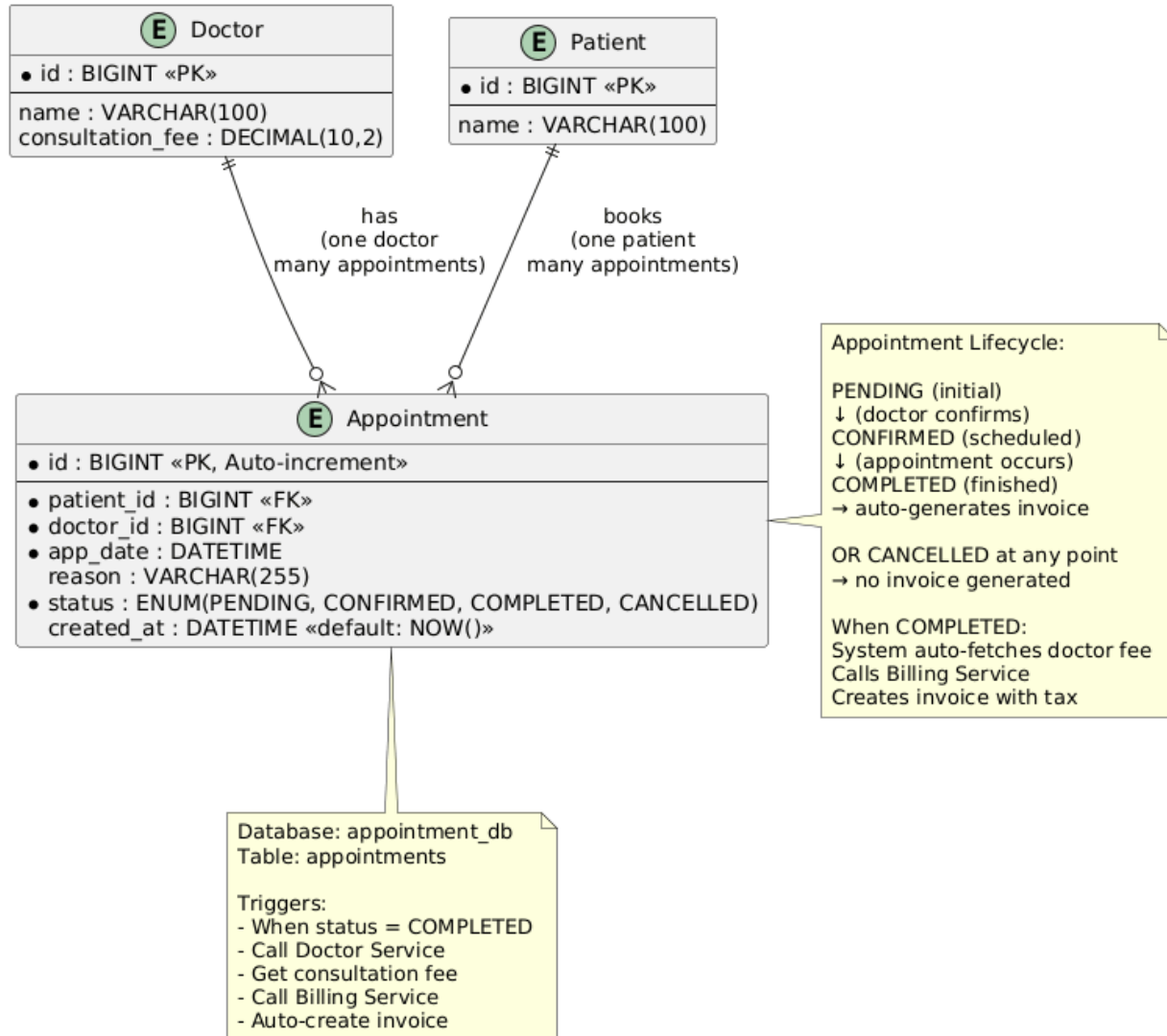
HMS Database - Tables & Columns Summary



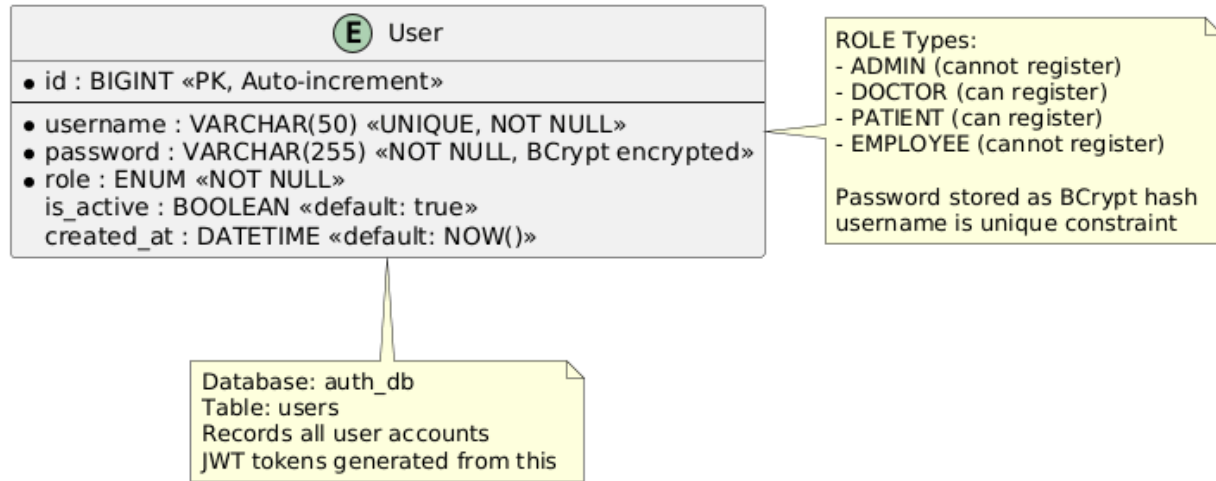
Patient Service - Patient Entity



Appointment Service - Appointment Entity



Auth Service - User Entity & Relationships



12. Conclusion

Based on the provided models, the system demonstrates strong integration between microservices architecture, design patterns, and AOP logging. The use of Docker ensures consistent deployment across environments, while Eureka and API Gateway provide robust service discovery and routing.